PEMROGRAMAN BERORIENTASI OBJEK 2026

UML *Class Diagram*

Bahasa gambar standar untuk memetakan kelas beserta hubungannya, sebelum dan sesudah kode ditulis

PERTEMUAN

06

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Menjelaskan apa itu **UML** dan posisi diagram kelas di antara jenis diagram UML lainnya.
2. Membaca dan menulis kotak kelas lengkap: visibilitas (+ - # ~), tipe data, parameter, nilai kembalian, dan anggota *static*.
3. Membedakan **enam jenis relasi** dari bentuk garisnya: asosiasi, agregasi, komposisi, dependensi, generalisasi, dan realisasi.
4. Membaca dan menuliskan **multiplisitas** serta **nama peran** pada sebuah relasi.
5. Menerjemahkan diagram menjadi kode (*forward engineering*) dan kode menjadi diagram (*reverse engineering*).
6. Menulis diagram kelas dengan sintaks **Mermaid**, lalu menampilkannya di Mermaid Live Editor, VS Code, atau GitHub.

Mengingat Kembali



Dari pertemuan sebelumnya

- **Dependensi**: memakai kelas lain sebentar saja, misalnya lewat parameter *method*.
- **Asosiasi**: objek mengenal objek lain dan menyimpan referensinya di atribut.
- **Agregasi** dan **komposisi**: hubungan punya; bagian komposisi dibuat oleh keseluruhan dan ikut berakhir bersamanya.
- **Multiplisitas** (`1`, `0..1`, `*`, `1..*`) menjawab pertanyaan "berapa banyak".

Agenda Hari Ini

1. Peta sebelum membangun: apa itu UML?
2. Anatomi kotak kelas dan visibilitas
3. Enam garis relasi
4. Multiplisitas dan nama peran
5. Mermaid, *forward*, dan *reverse engineering*
6. Studi kasus KedaiKita

BAGIAN 1

1 Peta Sebelum Membangun

Mengapa pemrogram butuh gambar, dan apa itu UML.

Dimas Butuh Peta

Analogi

- Dimas baru bergabung dengan tim KedaiKita. Siapa yang membuat `ItemPesanan`? Jika pesanan dihapus, apakah pelanggannya ikut hilang?
- Pertama kali ke Samarinda dan ingin ke Tepian Mahakam? Kamu membuka **peta**, bukan menyusuri setiap jalan.
- Arsitek menggambar **denah** sebelum tukang membangun. Mengubah denah jauh lebih murah daripada membongkar tembok.

Dunia nyata	Dunia PBO
Peta kota	**Diagram kelas** (*class diagram*)
Peta tidak menampilkan setiap batu di jalan	Diagram tidak menampilkan setiap baris kode
Denah sebelum membangun rumah	Rancangan sebelum menulis kode

Apa Itu UML?

UML (*Unified Modeling Language*)

Bahasa pemodelan visual yang **standar** untuk menggambarkan, merancang, dan mendokumentasikan perangkat lunak.

- **Bukan** bahasa pemrograman: UML tidak bisa dijalankan. Simbolnya disepakati bersama, mirip rambu lalu lintas.
- Disatukan oleh Booch, Rumbaugh, dan Jacobson, lalu diadopsi **OMG** sebagai standar pada 1997.
- Diagram kelas termasuk diagram struktur dan paling sering dipakai dalam PBO: **satu kotak = satu kelas**.

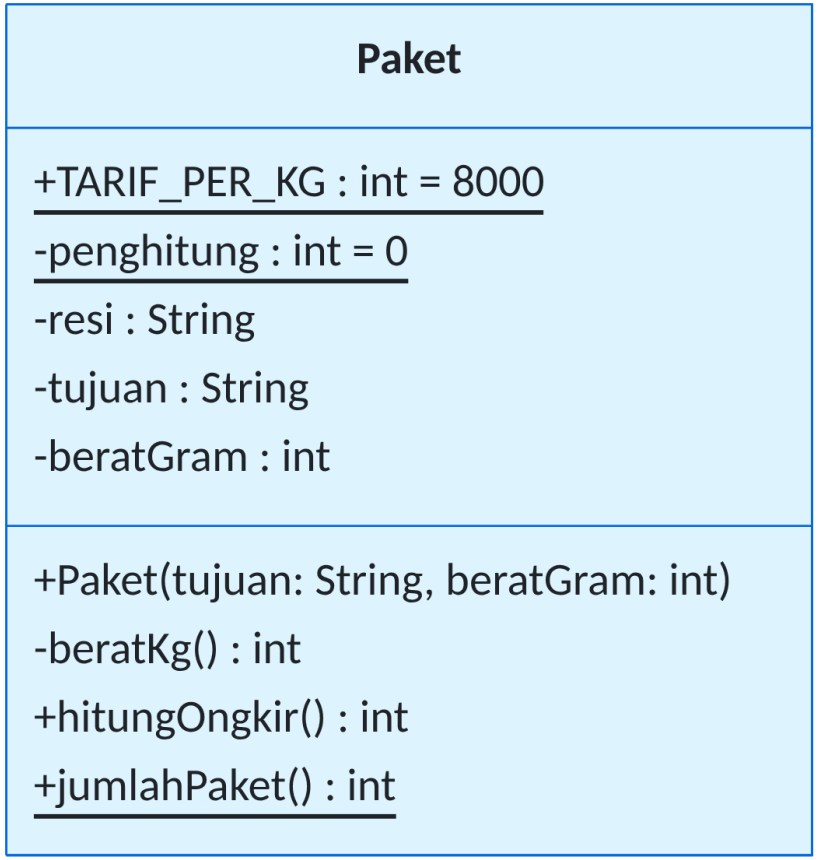
BAGIAN 2

2

Anatomi Kotak Kelas

Nama, atribut, dan *method*, ditambah visibilitas dan tipe.

Kotak Kelas Lengkap: Kelas `Paket`



Baris bergaris bawah adalah anggota *static* (milik kelas). Di Mermaid, barisnya diakhiri $.

Cara Membaca Baris Kotak Kelas

Baris di kotak	Cara membacanya
`-resi : String`	atribut **privat** bernama `resi`, bertipe teks
`+TARIF_PER_KG : int = 8000` (bergaris bawah)	atribut **publik** milik **kelas** (*static*), nilai awalnya 8000
`+Paket(tujuan: String, beratGram: int)`	**konstruktor**: namanya sama dengan nama kelas, tanpa tipe kembalian
`-beratKg() : int`	*method* privat tanpa parameter, mengembalikan `int`
`+jumlahPaket() : int` (bergaris bawah)	*method* milik kelas (*static*)

Pola: `visibilitas nama : Tipe` untuk atribut dan `visibilitas nama(parameter: Tipe) : TipeKembalian` untuk *method*.

Visibilitas: Siapa Boleh Mengakses?

Simbol	Siapa yang boleh mengakses	Python	Java
+ *public*	semua kelas (teras)	nama biasa: `harga`	`public`
- *private*	hanya kelas itu sendiri (kamar tidur)	awalan _: `_harga` (kesepakatan)	`private`
# *protected*	kelas itu dan kelas anaknya (ruang keluarga)	awalan _ juga (kesepakatan)	`protected` (juga terbuka untuk satu paket)
~ *package*	kelas-kelas dalam satu paket (taman kompleks)	tidak ada padanan	tanpa kata kunci (*default*)

Kebiasaan baik dari Pertemuan 04 tetap berlaku: atribut umumnya `-`, *method* layanan `+`, dan *method* pembantu internal `-`.

Kotak `Paket` Menjadi Kode

Baris bergaris bawah menjadi anggota milik kelas: atribut kelas dan `@classmethod` di Python, `static` di Java.

Python python/contoh/contoh01_kotak_kelas.py

```python
class Paket:
    TARIF_PER_KG = 8000
    _penghitung = 0

    def __init__(self, tujuan: str, berat_gram: int):
        Paket._penghitung += 1
        self._resi = f"KRM{Paket._penghitung:03d}"
        # ... (dilewati)
    # ... (dilewati)
    def _berat_kg(self) -> int:
        return (self._berat_gram + 999) // 1000
    # ... (dilewati)
    @classmethod
    def jumlah_paket(cls) -> int:
        return cls._penghitung
```

Java java/contoh/Contoh01KotakKelas.java

```java
class Paket {
    public static final int TARIF_PER_KG = 8000;
    private static int penghitung = 0;
    // ... (dilewati)
    public Paket(String tujuan, int beratGram) {
        penghitung++;
        this.resi = String.format("KRM%03d", penghitung);
        // ... (dilewati)
    }
    // ... (dilewati)
    private int beratKg() {
        return (beratGram + 999) / 1000;
    }
    // ... (dilewati)
    public static int jumlahPaket() {
        return penghitung;
    }
}
```

BAGIAN 3

3

Garis Relasi dan Multiplisitas

Bentuk garis, bentuk ujungnya, dan angka di kedua ujungnya.

Enam Garis Relasi UML

RELASI	NOTASI UML (A di kiri, B di kanan)	SINTAKS MERMAID
Asosiasi “mengenal”		`Mahasiswa --> DosenWali`
Agregasi “punya, longgar”		`Tim o-- Pemain`
Komposisi “punya, erat”		`Gedung *-- Lantai`
Dependensi “memakai sebentar”		`Motor ..> SPBU`
Generalisasi “B adalah A”		`Hewan <|-- Kucing`
Realisasi “B memenuhi kontrak A”		`DapatDicetak <|.. Laporan`

Baca tiga hal: garis penuh atau putus-putus, bentuk ujung, dan sisi tempelnya. Belah ketupat di sisi keseluruhan; segitiga di sisi induk.

Dependensi: Cukup Lewat Parameter

Di kotak `Motor` tidak ada atribut bertipe `SPBU`. `SPBU` hanya datang sebagai parameter: `Motor ..> SPBU`.

Python python/contoh/contoh05_dependensi.py

```python
class Motor:
    # ... (dilewati)
    def __init__(self, plat: str, kapasitas_tangki: int):
        self._plat = plat
        self._kapasitas_tangki = kapasitas_tangki
        self._isi_tangki = 0

    def isi_bensin(self, spbu: SPBU, liter: int) -> int:
        # ... (dilewati)
        biaya = liter_masuk * spbu.harga_per_liter
        # ... (dilewati)
        return biaya
# ... di dalam main()
total += motor.isi_bensin(spbu_juanda, 3)
total += motor.isi_bensin(spbu_pahlawan, 4)
```

Java java/contoh/Contoh05Dependensi.java

```java
class Motor {
    private String plat;
    private int kapasitasTangki;
    private int isiTangki;
    // ... (dilewati)
    public int isiBensin(SPBU spbu, int liter) {
        // ... (dilewati)
        int biaya = literMasuk * spbu.getHargaPerLiter();
        // ... (dilewati)
        return biaya;
    }
    // ... (dilewati)
}
// ... di dalam main
total += motor.isiBensin(spbuJuanda, 3);
total += motor.isiBensin(spbuPahlawan, 4);
```

Multiplisitas: Berapa Banyak?

Tulisan	Arti	Contoh	Wujud di kelas A
`1`	tepat satu	setiap pesanan milik tepat satu pelanggan	atribut bertipe B yang **wajib** diisi, biasanya lewat konstruktor
`0..1`	nol atau satu	pesanan ojek punya nol atau satu pengemudi	atribut yang boleh kosong (`None`/`null`)
`*`	nol atau lebih	pelanggan boleh punya banyak pesanan, atau belum punya	*list* berisi objek B
`1..*`	minimal satu	gedung punya minimal satu lantai	*list* yang dijamin kode berisi minimal satu objek

Angka yang dipakai selalu yang di **ujung seberang**: `Pesanan "*" --> "1" Pelanggan` berarti setiap pesanan punya satu pelanggan.

Asosiasi Berarah: `1` dan `0..1` di Kode

Penumpang wajib ada sejak pesanan lahir (`1`). Pengemudi boleh menyusul (`0..1`).

Python python/contoh/contoh02_asosiasi.py

```python
class PesananOjek:
    # ... (dilewati)
    def __init__(self, penumpang: Penumpang, tujuan: str):
        self._penumpang = penumpang
        self._tujuan = tujuan
        self._pengemudi: Pengemudi | None = None

    def tetapkan_pengemudi(self, pengemudi: Pengemudi):
        self._pengemudi = pengemudi
```

Java java/contoh/Contoh02Asosiasi.java

```java
class PesananOjek {
    private Penumpang penumpang;
    private String tujuan;
    private Pengemudi pengemudi;

    public PesananOjek(Penumpang penumpang, String tujuan) {
        this.penumpang = penumpang;
        this.tujuan = tujuan;
        this.pengemudi = null;
    }

    public void tetapkanPengemudi(Pengemudi pengemudi) {
        this.pengemudi = pengemudi;
    }
    // ... (dilewati)
}
```

BAGIAN 4

4

Mermaid serta *Forward* dan *Reverse Engineering*

Menulis diagram sebagai teks, lalu berpindah dari diagram ke kode dan sebaliknya.

Menulis Diagram Kelas dengan Mermaid

Yang ingin digambar	Tulis di Mermaid	Tampil sebagai
Kelas dengan isi	`class Menu { ... }`	kotak tiga bagian
Method	`+hitungTotal(jumlah: int) int`	`+hitungTotal(jumlah: int) : int`
Anggota *static*	akhiri baris dengan `$`	bergaris bawah
Anggota abstrak	akhiri baris dengan `*`	miring
Tipe generik	`List~Menu~`	`List<Menu>`
Relasi lengkap	`Pesanan "1" *-- "*" ItemPesanan : daftarItem`	garis, multiplisitas, dan label

Coba di Mermaid Live Editor (mermaid.live): tempel teks mulai dari baris `classDiagram`.

Forward dan *Reverse Engineering*

Forward engineering

- Membuat kode **dari** diagram: rancang dulu, baru tulis kodenya.
- Seperti membangun rumah baru dari denah arsitek.
- Setiap kotak menjadi kelas; setiap garis menjadi atribut, *list*, atau parameter.

Reverse engineering

- Membuat diagram **dari** kode yang sudah ada: pahami dulu, baru ubah.
- Seperti mengukur rumah tua yang denahnya hilang, lalu menggambar ulang denahnya.
- Atribut bertipe kelas lain menjadi garis; kelas yang hanya jadi parameter menjadi garis putus-putus.

Dari Diagram ke Kode Python dan Java

Elemen diagram	Python	Java
`-harga : int`	`self._harga = harga` di `__init__`	`private int harga;`
`+hitungTotal(jumlah: int) int`	`def hitung_total(self, jumlah: int) -> int:`	`public int hitungTotal(int jumlah)`
`A --> "0..1" B`	`self._b: B | None = None`	*field* `private B b;` yang boleh `null`
`A o-- "*" B`	*list*; objek `B` diterima lewat parameter	`List<B>`; objek `B` diterima lewat parameter
`A *-- "*" B`	*list*; objek `B` dibuat di dalam `A`	`List<B>`; `new B(...)` di dalam `A`
`A ..> B`	`B` hanya parameter atau variabel lokal	`B` hanya parameter atau variabel lokal

Padanan tipe di Python: `String` menjadi `str`, `boolean` menjadi `bool`, dan `List~Menu~` menjadi `list[Menu]`.

STUDI KASUS



Studi Kasus KedaiKita

Reverse engineering kode KedaiKita, lalu merencanakan Pertemuan 07.

Atribut Pesanan Menjadi Garis

Studi kasus KedaiKita

Setiap atribut yang tipenya kelas lain menjadi satu garis. Siapa yang membuat objeknya menentukan jenis garisnya.

Python python/studi-kasus/pesanan.py

```python
class Pesanan:
    # ... (dilewati)
    _nomor_terakhir = 0

    def __init__(self, pelanggan: Pelanggan):
        Pesanan._nomor_terakhir += 1
        self._nomor = f"P{Pesanan._nomor_terakhir:03d}"
        self._pelanggan = pelanggan
        self._daftar_item: list[ItemPesanan] = []
    # ... (dilewati)
    def tambah_item(self, menu: Menu, jumlah: int):
        item = ItemPesanan(menu, jumlah)
        menu.kurangi_stok(jumlah)
        self._daftar_item.append(item)
```

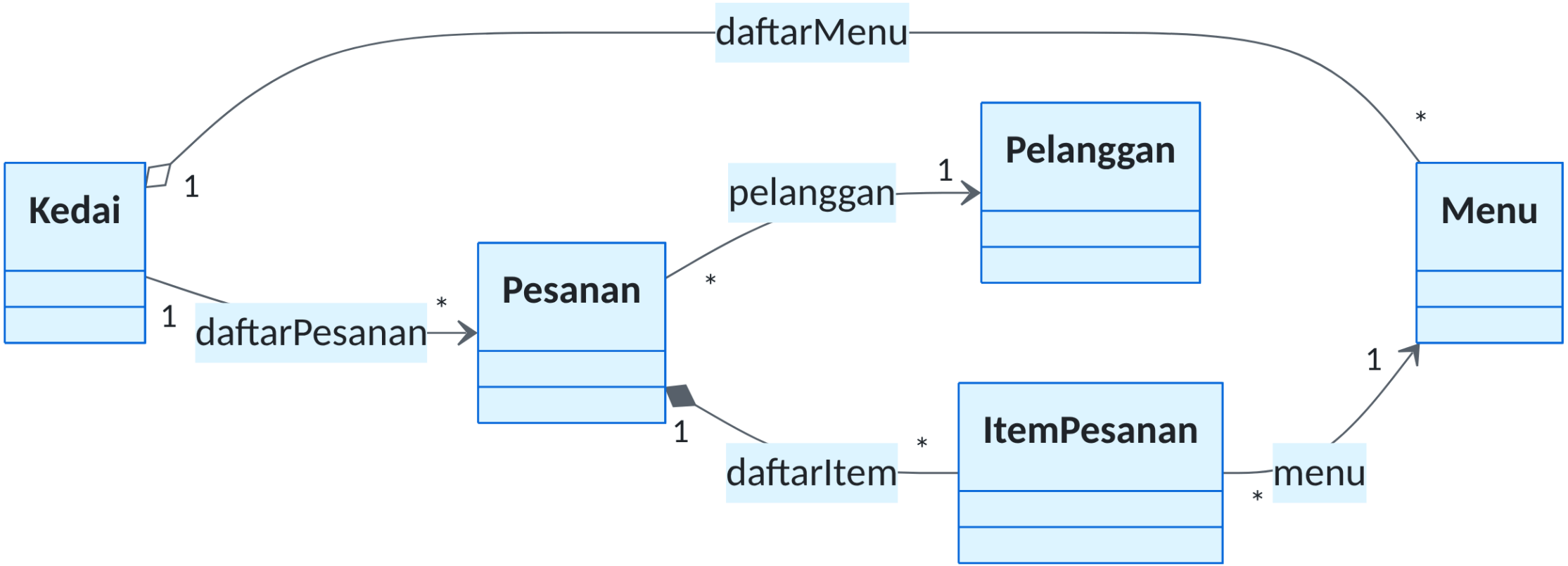
Java java/studi-kasus/Pesanan.java

```java
public class Pesanan {
    private static final int LEBAR_STRUK = 40;
    private static int nomorTerakhir = 0;

    private String nomor;
    private Pelanggan pelanggan;
    private List<ItemPesanan> daftarItem;
    // ... (dilewati)
    public void tambahItem(Menu menu, int jumlah) {
        ItemPesanan item = new ItemPesanan(menu, jumlah);
        menu.kurangiStok(jumlah);
        daftarItem.add(item);
    }
    // ... (dilewati)
}
```

Peta KedaiKita: Garis dari Atribut

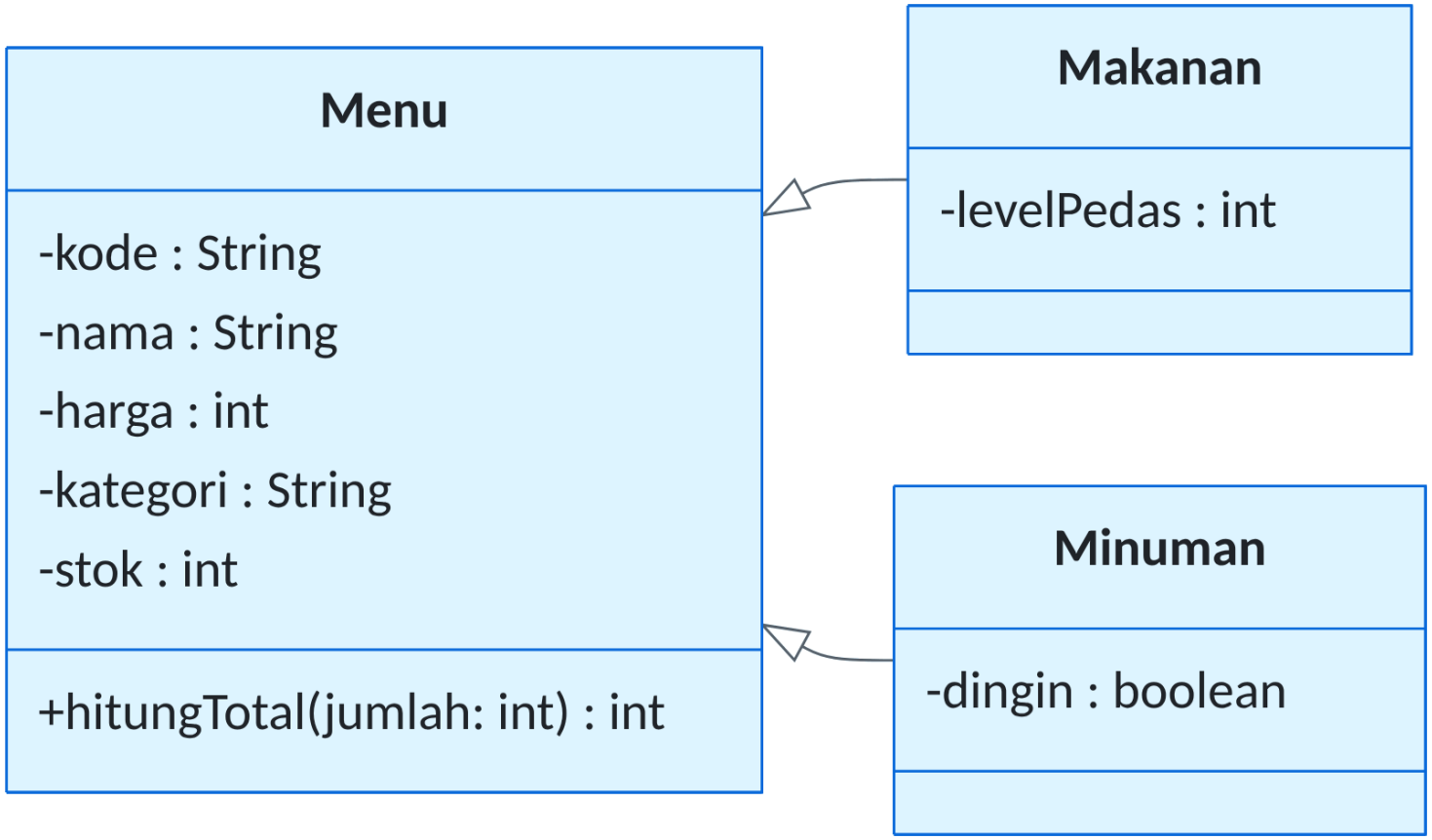
Studi kasus KedaiKita



Label garis adalah nama peran, sama dengan nama atribut di kode. Diagram lengkap beserta dependensinya ada di halaman konsep.

Merencanakan Pertemuan 07: `Makanan` dan `Minuman`

Studi kasus KedaiKita



Kotak anak hanya berisi anggota **baru**; isi `Menu` diwarisi. Konstruktor kedua kelas anak tidak digambar di sini.

Miskonsepsi yang Sering Terjadi

Anggapan keliru

"`Makanan <|-- Menu` berarti makanan adalah menu."

"Belah ketupat boleh ditaruh di ujung mana saja, misalnya `ItemPesanan *-- Pesanan`."

"Setiap kelas lain yang disebut di dalam kode digambar sebagai asosiasi."

Yang benar

Segitiga menempel pada **induk**: tulis `Menu <|-- Makanan`. Penulisan `Makanan <|-- Menu` justru berarti menu adalah makanan.

Belah ketupat selalu menempel pada **keseluruhan**. Yang benar adalah `Pesanan *-- ItemPesanan`.

Asosiasi hanya untuk objek yang **disimpan di atribut**. Kelas yang hanya muncul sebagai parameter atau variabel lokal adalah dependensi (`..>`).

Kuis Cepat

Di sebuah kotak kelas tertulis `#biayaAdmin : int`. Artinya ...

A atribut publik bertipe `int`

B atribut milik kelas (*static*) bernama `biayaAdmin`

C atribut *protected*: boleh diakses kelas itu sendiri dan kelas anaknya

D atribut yang hanya boleh diakses kelas-kelas dalam satu paket

Kuis Cepat: Jawaban

Di sebuah kotak kelas tertulis `#biayaAdmin : int`. Artinya …

A. atribut publik bertipe `int`

B. atribut milik kelas (*static*) bernama `biayaAdmin`

✓ C. atribut *protected*: boleh diakses kelas itu sendiri dan kelas anaknya

D. atribut yang hanya boleh diakses kelas-kelas dalam satu paket

Jawaban: C

`#` berarti *protected*. Publik ditulis `+` (A), dan akses satu paket ditulis `~` (D). Anggota *static* tidak ditandai simbol di depan, tetapi dengan garis bawah (di Mermaid: `$` di akhir baris) sehingga B salah.

Kuis Cepat

Perhatikan relasi `Pesanan "1" *-- "*" ItemPesanan`.
Pernyataan yang benar adalah ...

A Setiap pesanan berisi nol atau lebih item, dan setiap item milik tepat satu pesanan

B Setiap item boleh dimiliki banyak pesanan sekaligus

C Setiap pesanan berisi tepat satu item

D Pesanan dan item bisa hidup terpisah tanpa saling bergantung

Kuis Cepat: Jawaban

Perhatikan relasi `Pesanan "1" *-- "*" ItemPesanan`. Pernyataan yang benar adalah ...

✓ Setiap pesanan berisi nol atau lebih item, dan setiap item milik tepat satu pesanan

B. Setiap item boleh dimiliki banyak pesanan sekaligus

C. Setiap pesanan berisi tepat satu item

D. Pesanan dan item bisa hidup terpisah tanpa saling bergantung

Jawaban: A

Dibaca dari `Pesanan`, angka di ujung seberang adalah `*`: satu pesanan berisi nol atau lebih item.
Dibaca dari `ItemPesanan`, angkanya `1`: setiap item milik tepat satu pesanan, jadi B dan C salah.
Belah ketupat penuh berarti komposisi: item tidak berarti tanpa pesanannya sehingga D juga salah.

Ringkasan

- **UML** adalah bahasa pemodelan visual standar (OMG, 1997); diagram kelas paling sering dipakai dalam PBO.
- Kotak kelas: `visibilitas nama : Tipe` untuk atribut dan `visibilitas nama(parameter : Tipe) : TipeKembalian` untuk *method*.
- Visibilitas: + *public*, - *private*, # *protected*, ~ *package*. Anggota *static* digarisbawahi.
- Enam relasi: `-->`, `o--`, `*--`, `..>`, `<|--`, dan `<|..` (realisasi). Belah ketupat di keseluruhan, segitiga di induk.
- **Multiplisitas** dibaca dari angka di ujung seberang; **nama peran** biasanya sama dengan nama atribut.
- **Mermaid** mengubah teks menjadi diagram: `$` untuk *static*, `*` untuk abstrak, `List~Menu~` untuk tipe generik.
- ***Forward engineering***: kode dari diagram. ***Reverse engineering***: diagram dari kode.

Langkah Berikutnya

- Jalankan contoh 01—06 dan cocokkan setiap kodenya dengan diagram di halaman praktik.
- Coba tulis diagrammu sendiri di Mermaid Live Editor, lalu kerjakan latihan dan kuis 10 soal.

Buka bahan pertemuan ini di GitHub:

- Konsep
- Praktik Python
- Praktik Java
- Jembatan Python ↔ Java
- Latihan
- Kuis 10 soal

Pindai untuk membuka folder pertemuan ini di GitHub

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-06-UML-Class-Diagram